UML DESIGN & MODELING PACKAGE

DocuQuery AI: Enterprise Retrieval-Augmented Generation Platform

Course: Java Project Based Learning (PBL) — 5th Semester
Institution: Arya College of Engineering & I.T., Jaipur (Dept. of AI&DS - Artificial Intelligence & Data Science)
Mentor: Er. Ram Babu Buri
Team Members: Shaurya Dixit, Yashvardhan Singh Chauhan, Vaibhav Tank, Rajveer Singh Ranawat, Yashpal Singh Bhati
Data Corpus: Kaggle Academic Research Paper Corpus

TABLE OF CONTENTS

1. Use Case Diagram & Actor Descriptions
2. Class Diagram (Domain Models & Services)
3. Sequence Diagram (End-to-End RAG Query Flow)
4. Activity Diagram (Document Ingestion & Chunking Pipeline)
5. Component & Deployment Diagram
6. Entity-Relationship (ER) Diagram (Neon PostgreSQL Database)

---

1. USE CASE DIAGRAM

```
flowchart LR
    Student(("Student / User"))
    Researcher(("Researcher"))
    Admin(("Administrator"))

    subgraph DocuQueryAI["DocuQuery AI System Boundary"]
        UC1["Register & Authenticate (JWT)"]
        UC2["Upload Documents (PDF, DOCX)"]
        UC3["Ask Natural Language Questions"]
        UC4["View Streaming Answer & Citations"]
        UC5["Inspect Source Documents & Excerpts"]
        UC6["Export Conversation History"]
        UC7["Manage Knowledge Base (Re-index/Delete)"]
        UC8["View RAG Triad & System Metrics"]
        UC9["Configure LLM & Vector Parameters"]
    end

    Student --> UC1
    Student --> UC3
    Student --> UC4
    Student --> UC5
    Student --> UC6

    Researcher --> UC1
    Researcher --> UC2
    Researcher --> UC3
    Researcher --> UC4
    Researcher --> UC5
    Researcher --> UC6

    Admin --> UC1
    Admin --> UC2
    Admin --> UC7
    Admin --> UC8
    Admin --> UC9
```

Actor Roles:

- **Student**: General user who authenticates, asks natural-language questions, inspects source citation badges, and downloads chat histories.
- **Researcher**: Contributor who uploads research papers, books, and curriculum notes into the shared workspace.
- **Administrator**: System administrator with privileges to monitor vector capacity, view RAG Triad telemetry, and purge documents.

---

2. CLASS DIAGRAM

```
classDiagram
    class UserEntity {
        -UUID id
        -String username
        -String email
        -String password
        -String role
        -Instant createdAt
        +getId() UUID
        +getUsername() String
        +getRole() String
    }

    class DocumentEntity {
        -UUID id
        -String filename
        -String contentType
        -Long fileSize
        -Integer totalPages
        -Integer totalChunks
        -DocumentStatus status
        -Instant createdAt
        +getId() UUID
        +getStatus() DocumentStatus
    }

    class DocumentChunkEntity {
        -UUID id
        -Integer chunkIndex
        -Integer pageNumber
        -String content
        -String embedding
        -Integer tokenCount
        -Instant createdAt
        +getId() UUID
        +getContent() String
        +getEmbedding() String
    }

    class RagOrchestrationService {
        -DocumentChunkRepository chunkRepository
        -int defaultTopK
        -double similarityThreshold
        +processQuery(ChatRequest request) ChatResponse
        -generateMockVectorString(int dim) String
    }

    class DocumentIngestionService {
        -DocumentRepository documentRepository
        -DocumentChunkRepository chunkRepository
        -Tika tika
        +ingestDocumentAsync(MultipartFile file) CompletableFuture
        -recursiveSplitText(String text, int size, int overlap) List
    }

    class EvaluationService {
        +getSystemMetrics() Map
```

```
    }

    DocumentEntity "1" *-- "many" DocumentChunkEntity : owns
    DocumentIngestionService ..> DocumentEntity : manages
    RagOrchestrationService ..> DocumentChunkEntity : queries
    EvaluationService ..> RagOrchestrationService : monitors
```

---

3. SEQUENCE DIAGRAM (RAG Query Execution)

```
sequenceDiagram
    autonumber
    actor User as User (Desktop App)
    participant Electron as Electron / React UI
    participant Gateway as Spring Security (JWT)
    participant RAG as RagOrchestrationService
    participant Neon as Neon PostgreSQL (pgvector)
    participant LLM as LLM Inference Service

    User->>Electron: Types natural language question
    Electron->>Gateway: POST /api/rag/chat (Bearer Token + JSON)
    Gateway->>Gateway: Validate JWT & Enforce Rate Limit (Bucket4j)
    Gateway->>RAG: Forward authorized query
    RAG->>RAG: Generate query vector embedding (1536 float dimensions)
    RAG->>Neon: SELECT * FROM document_chunks ORDER BY embedding <=> :vec LIMIT 4
    Neon-->>RAG: Returns Top-4 Chunks + Metadata (Page, Source)
    RAG->>RAG: Assemble System Prompt + Inject Context + Constraints
    RAG->>LLM: Send Grounded Prompt (Context + Question)

    loop Streaming Server-Sent Events
        LLM-->>RAG: Emits token chunk
        RAG-->>Electron: SSE event: data: {"token": "The", "citations": []}
        Electron-->>User: Render token in chat bubble
    end

    RAG-->>Electron: SSE event: data: {"done": true, "citations": [{"docId": "...", "page": 4}]}
    Electron-->>User: Render interactive Citation Badge
```

---

4. ACTIVITY DIAGRAM (Document Chunking Pipeline)

```
flowchart TD
    Start([User uploads Document]) --> ValidateFormat{Is format PDF, DOCX, TXT?}
    ValidateFormat -- No --> Reject([Reject with HTTP 400 Bad Request])
    ValidateFormat -- Yes --> CheckSize{Is file <= 25MB?}
    CheckSize -- No --> RejectSize([Reject with HTTP 413 Payload Too Large])
    CheckSize -- Yes --> AsyncQueue[Dispatch Async Processing Task]
    AsyncQueue --> TikaParse[Extract Raw Text via Apache Tika]
    TikaParse --> SentenceSplit[Parse Sentence Boundaries with BreakIterator]
    SentenceSplit --> ChunkLoop[Slice into 500-token chunks with 50-token overlap]
    ChunkLoop --> Embeddings[Generate 1536-dim Vector Embeddings]
    Embeddings --> NeonInsert[Batch INSERT into document_chunks table]
    NeonInsert --> HNSWBuild[Update HNSW Vector Index]
    HNSWBuild --> Notify[Emit Desktop Notification: 'Document Indexed']
    Notify --> End([Indexing Completed])
```

---

5. COMPONENT & DEPLOYMENT DIAGRAM

```
flowchart TB
    subgraph ClientMachine["Client Machine (Windows 10/11)"]
        subgraph ElectronDesktop["Electron 30 Runtime"]
            Main["Electron Main Process (IPC & Window Lifecycle)"]
            Preload["Sandboxed Preload Script"]
            ReactUI["React 18 SPA (TypeScript + Tailwind CSS)"]
        end
    end

    subgraph ServerCloud["Backend Application Server"]
        subgraph SpringMicroservice["Java 21 / Spring Boot 3.3"]
            SecFilter["Spring Security 6 (JWT + BCrypt)"]
            RateLimit["Bucket4j Token Bucket"]
            IngestEng["Document Ingestion Engine (Apache Tika)"]
            RAGEng["RAG Orchestration Service (Spring AI)"]
            TelemetryEng["RAG Triad Evaluator & Telemetry"]
            HikariPool["HikariCP Connection Pool"]
        end
    end

    subgraph NeonCloud["Neon Serverless PostgreSQL Cloud"]
        PgBouncer["PgBouncer Transaction Pooler"]
        PostgresCore[("PostgreSQL 16 Database Engine")]
        HNSWIndex[("pgvector HNSW Cosine Index")]
    end

    ReactUI --> Preload --> Main
    ReactUI -- "HTTPS / REST & SSE" --> SecFilter
    SecFilter --> RateLimit --> IngestEng & RAGEng & TelemetryEng
    IngestEng --> HikariPool
    RAGEng --> HikariPool
    TelemetryEng --> HikariPool
    HikariPool -- "TLS 1.3" --> PgBouncer --> PostgresCore
    PostgresCore --- HNSWIndex
```

---

6. ENTITY-RELATIONSHIP (ER) DIAGRAM

```
erDiagram
    USERS ||--o{ DOCUMENTS : "owns"
    DOCUMENTS ||--|{ DOCUMENT_CHUNKS : "contains"
    USERS ||--o{ AUDIT_LOGS : "generates"

    USERS {
        uuid id PK
        string username UK
        string email UK
        string password
        string role
        timestamp created_at
    }

    DOCUMENTS {
        uuid id PK
        uuid user_id FK
        string filename
        string content_type
        bigint file_size
        integer total_pages
        integer total_chunks
        string status
        timestamp created_at
    }

    DOCUMENT_CHUNKS {
        uuid id PK
        uuid document_id FK
        integer chunk_index
        integer page_number
        text content
        integer token_count
        vector_1536 embedding
        timestamp created_at
    }

    AUDIT_LOGS {
        uuid id PK
        uuid user_id FK
        string query_text
        integer tokens_used
        float latency_ms
        timestamp created_at
    }
```